

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1 Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Tasks to be Performed

1. Download and understand the dataset.
2. Perform Exploratory Data Analysis (EDA).
3. Preprocess the dataset.
4. Implement a Single Layer Perceptron from scratch.
5. Train the model using the perceptron learning rule.
6. Evaluate the classifier using standard performance metrics.
7. Analyze the effect of different learning rates.
8. Compare the implementation with Scikit-learn's Perceptron (optional).

3. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

4. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

5. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

6. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output

Dataset Dimensions: (1372, 5)

First five samples:

Index	Variance	Skewness	Curtosis	Entropy	Class
0	3.62160	8.6661	-2.8073	-0.44699	0
1	4.54590	8.1674	-2.4586	-1.46210	0
2	3.86600	-2.6383	1.9242	0.10645	0
3	3.45660	9.5228	-4.0112	-3.59440	0
4	0.32924	-4.4552	4.5718	-0.98880	0

Missing Values Summary:

Column	Missing Values
Variance	0
Skewness	0
Curtosis	0
Entropy	0
Class	0

Descriptive Statistics:

Statistic	Variance	Skewness	Curtosis	Entropy	Class
count	1372.000000	1372.000000	1372.000000	1372.000000	1372.000000
mean	0.433735	1.922353	1.397627	-1.191657	0.444606
std	2.842763	5.869047	4.310030	2.101013	0.497103
min	-7.042100	-13.773100	-5.286100	-8.548200	0.000000
25%	-1.773000	-1.708200	-1.574975	-2.413450	0.000000
50%	0.496180	2.319650	0.616630	-0.586650	0.000000
75%	2.821475	6.814625	3.179250	0.394810	1.000000
max	6.824800	12.951600	17.927400	2.449500	1.000000

Task 2: Exploratory Data Analysis

Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training

Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

7. Mandatory Plots

Plot	Purpose	Expected Inference
Feature Histograms	Distribution Analysis	Identify skewness and outliers.

Correlation Heatmap	Feature Relationship	Observe highly correlated features.
Scatter Plot	Class Distribution	Determine whether classes are approximately linearly separable.
Training Error vs Epoch	Learning Behaviour	Verify convergence of the perceptron.
Weight Evolution	Parameter Analysis	Observe stabilization of learned weights.
Bias Evolution	Parameter Analysis	Understand the role of the bias term.
Confusion Matrix	Performance Evaluation	Interpret TP, FP, TN and FN.
Learning Rate Comparison	Hyperparameter Study	Compare convergence behaviour for different learning rates.
Decision Boundary (Optional)	Visualization	Illustrate the separating hyperplane using two selected features.

Generated Plots and Observations

1. Feature Histograms

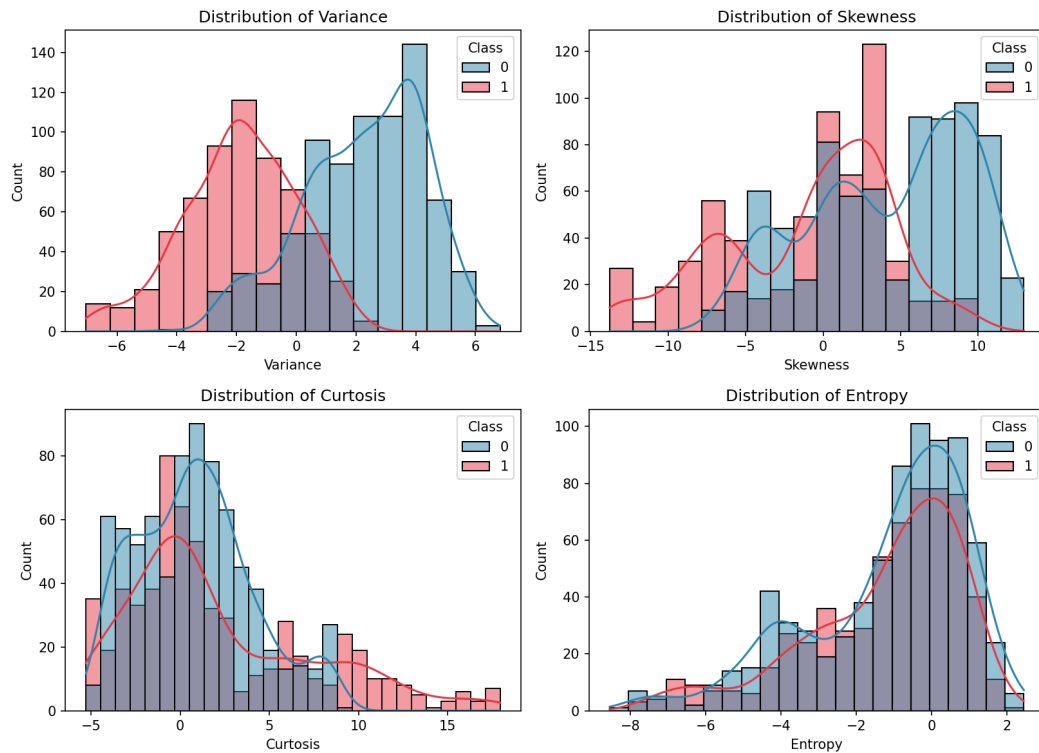


Figure 1: Feature Histograms of the Banknote Authentication Dataset

Observation:

Variance and skewness differ considerably between class 0 and class 1. Entropy overlaps quite a bit across both classes, making it very less informative.

2. Correlation Heatmap

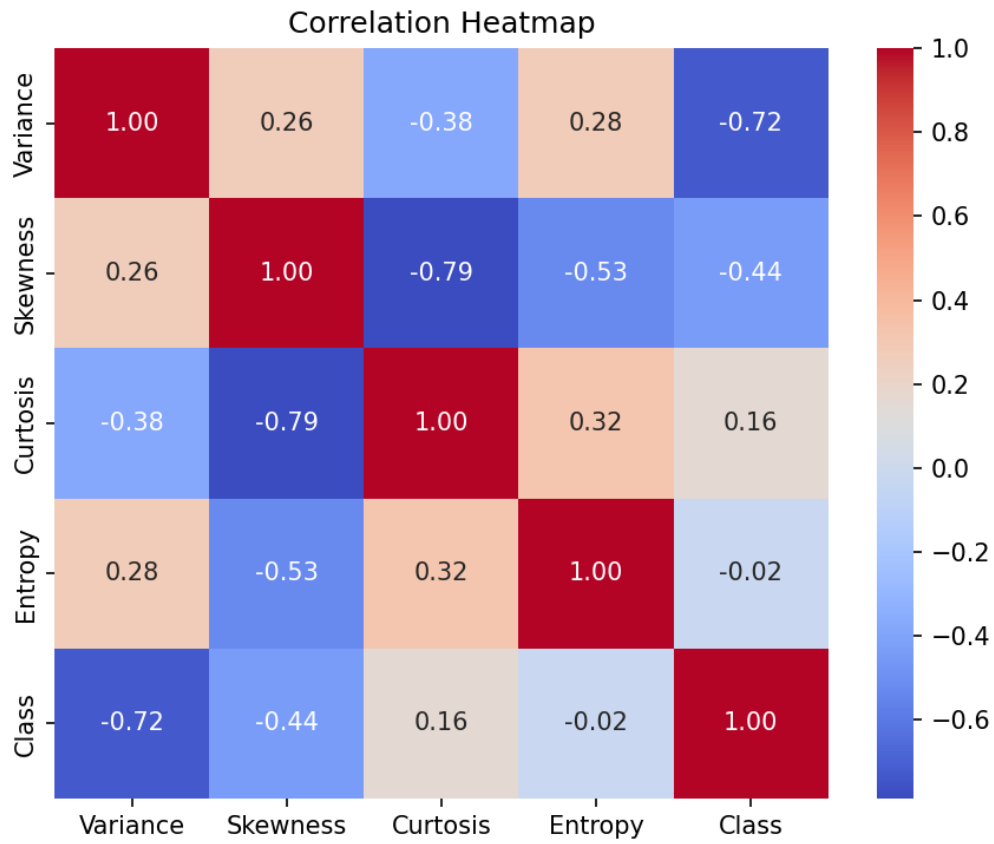


Figure 2: Correlation Heatmap of Dataset Features

Observation:

Variance has a strong negative correlation with target class, i.e. it is very informative for such classification. Skewness also shows decent magnitude of correlation.

3. Variance vs Skewness Scatter Plot

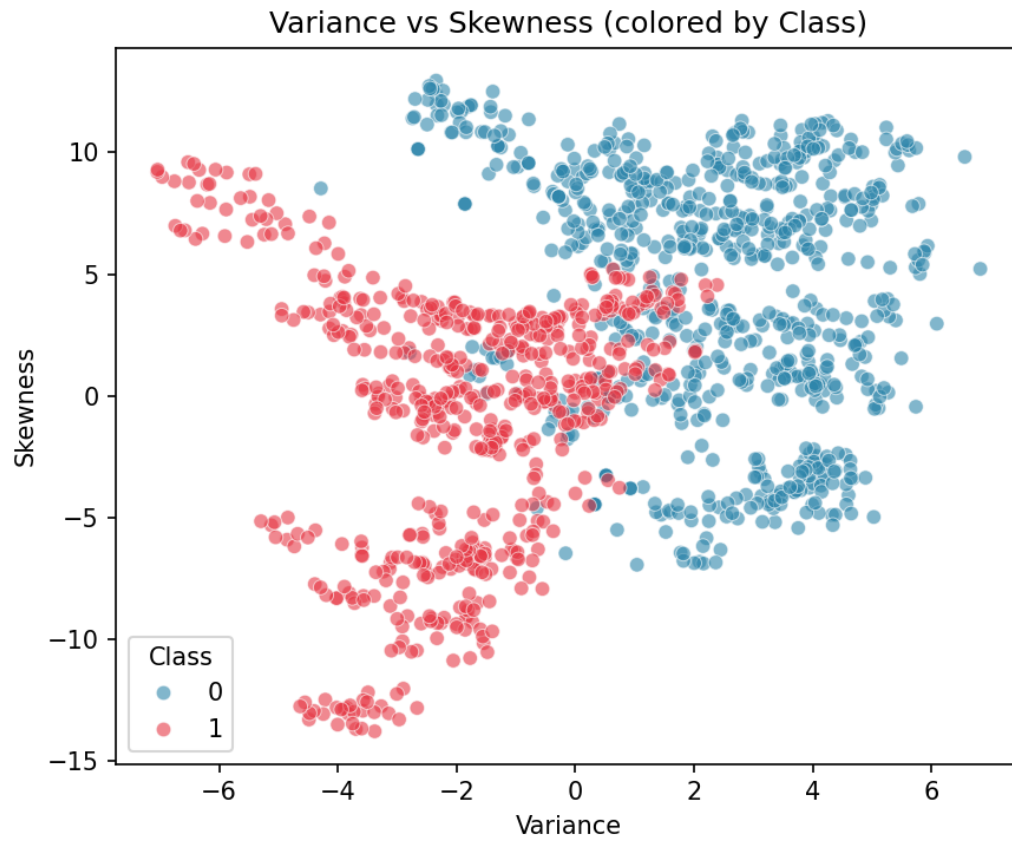


Figure 3: Scatter Plot of Variance versus Skewness

Observation:

Class 0 (authentic) is concentrated in the positive region, while class 1 (fake) is mostly negative with similar horizontal spread.

4. Training Error vs Epoch

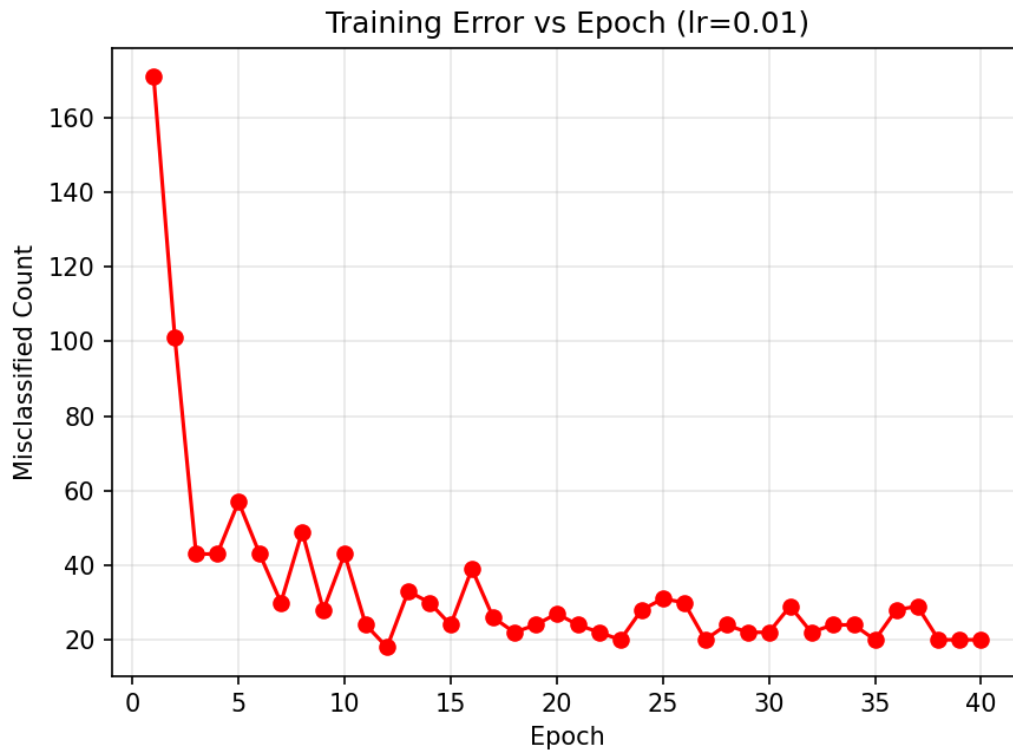


Figure 4: Training Error Across Epochs

Observation:

After the first few iterations, the training error drops significantly. Beyond 5 epochs, the error oscillates every now and then before settling down at around 20-30 misclassified samples.

5. Weight Evolution

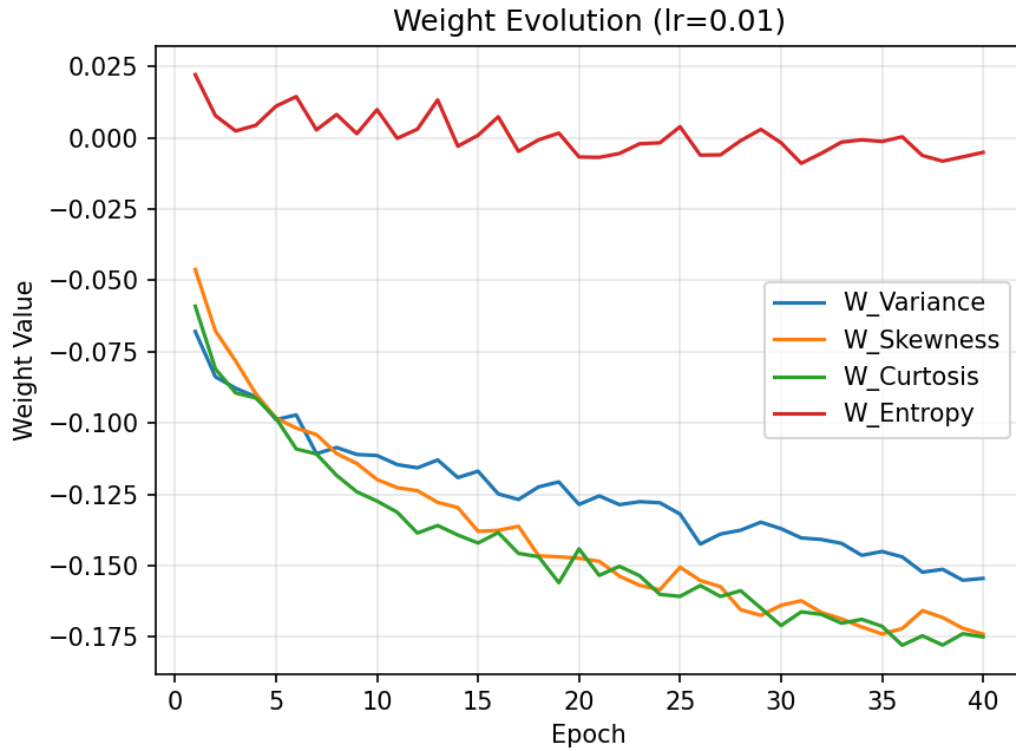


Figure 5: Evolution of Perceptron Weights

Observation:

Entropy reaches the neighbourhood of zero quickly and oscillates; signifying how little it contributes. The other 3 features steadily become more negative before eventually settling down.

6. Bias Evolution

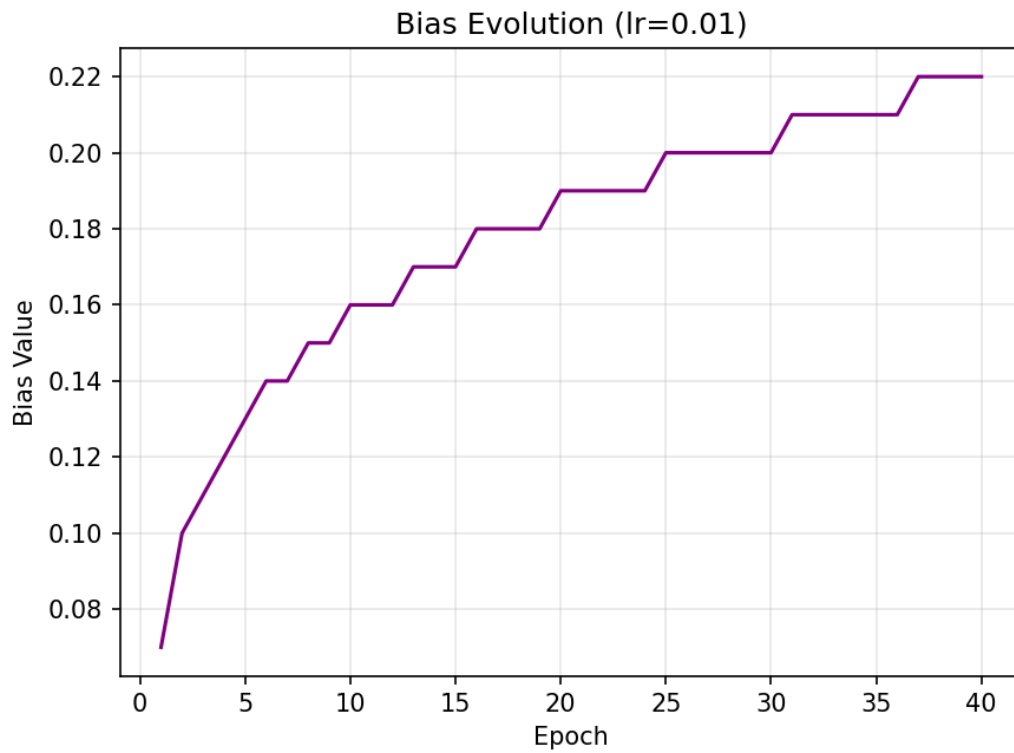


Figure 6: Evolution of Perceptron Bias

Observation:

Bias rises steadily before eventually settling at around 0.22. This is probably to account for the decision hyperplane.

7. Confusion Matrix

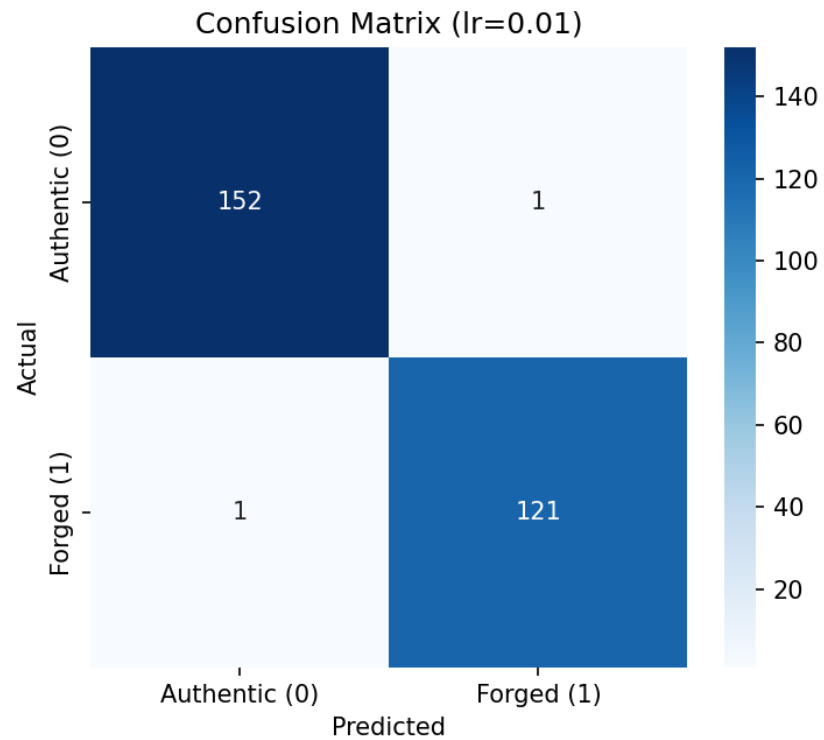


Figure 7: Confusion Matrix of the Perceptron Classifier

Observation:

Only 1 FP and FN each, signifying most training errors are near the boundary.

8. Learning Rate Comparison



Figure 8: Performance Comparison Across Different Learning Rates

Observation:

All 3 curves overlap exactly and give the same trajectory. Feature normalisation helps; and since this is a sample dataset, a magnitude difference of the order of 10 does not give drastic differences.

9. Weight Evolution by Learning Rate

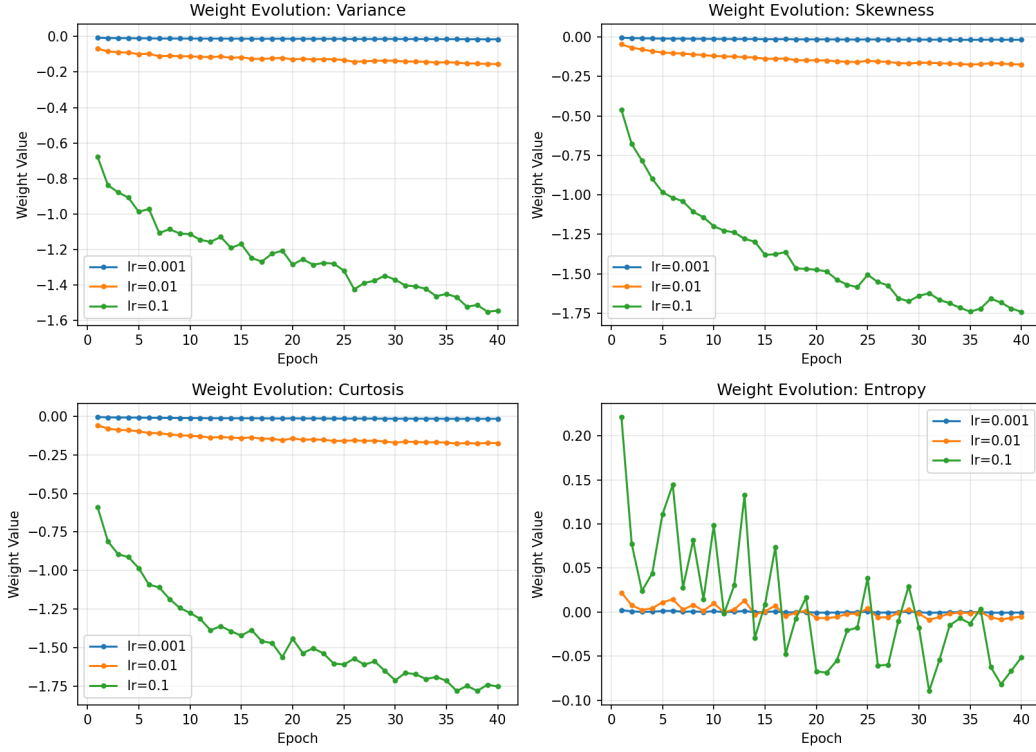


Figure 9: Weight Evolution for Different Learning Rates

Observation:

Except entropy, the other 3 features do not evolve much at learning rates of 0.001 and 0.01. However, they steadily decrease at 0.1 and converge. In the case of entropy, 0.001 and 0.01 prove to be fairly stable, while 0.1 results in wild oscillations.

8. Performance Tables

Training Summary

Dataset Size	1372
Train/Test Split	1097/275 (80/20)
Learning Rate	0.01
Epochs	40
Final Weights	[-0.1545, -0.1740, -0.1750, -0.0051]
Final Bias	0.22
Accuracy	0.9927
Precision	0.9918
Recall	0.9918
F1-score	0.9918

Epoch wise Learning (lr=0.01)

Epoch	Errors	Weight 1	Weight 2	Weight 3	Weight 4	Bias
1	171	-0.0679	-0.0462	-0.0590	0.0221	0.07
5	57	-0.0988	-0.0983	-0.0984	0.0111	0.13
10	43	-0.1114	-0.1198	-0.1274	0.0099	0.16
15	24	-0.1169	-0.1380	-0.1421	0.0009	0.17
20	27	-0.1286	-0.1474	-0.1441	-0.0067	0.19
25	31	-0.1319	-0.1506	-0.1608	0.0039	0.20
30	22	-0.1370	-0.1639	-0.1710	-0.0018	0.20
35	20	-0.1450	-0.1739	-0.1713	-0.0013	0.21
40	20	-0.1545	-0.1740	-0.1750	-0.0051	0.22

Metric	Value
Accuracy	0.9927
Precision	0.9918
Recall	0.9918
F1-score	0.9918

Confusion Matrix

$$\begin{bmatrix} 152 & 1 \\ 1 & 121 \end{bmatrix}$$

Learning Rate Comparison

Learning Rate	Epochs	Converged	Accuracy	Precision	Recall	F1-score
0.001	40	No	0.9927	0.9918	0.9918	0.9918
0.01	40	No	0.9927	0.9918	0.9918	0.9918
0.1	40	No	0.9927	0.9918	0.9918	0.9918

All 3 learning rates achieved identical classification performance on the banknote dataset. Although none of the models completely converged within 40 epochs, they reached very high accuracy and identical precision, recall and F1 scores.

9. Additional Tasks

1. Step vs Sigmoid Activation Function

The Step Activation Function produces binary outputs and is suitable for classical perceptrons. However, it is not differentiable in between and therefore cannot be used most of the time.

The Sigmoid Activation Function produces smooth outputs between 0 and 1 and is differentiable, making it far more usable.

2. Comparison with Scikit-learn's Perceptron

Metric	From Scratch	Scikit-learn
Accuracy	0.9927	0.9709
Precision	0.9918	0.9385
Recall	0.9918	1.0000
F1-score	0.9918	0.9683

The from scratch implementation slightly outperformed Scikit-learn's Perceptron in all aspects.

3. Learning Rate Analysis

The perceptron was trained using learning rates 0.001, 0.01 and 0.1. All three learning rates achieved pretty much identical classification performance. The dataset is not perfectly linearly separable and therefore none of the models completely converged within 40 iterations.

4. XOR Problem

A single layer perceptron cannot solve the XOR problem because XOR is not linearly separable. Since a perceptron can learn only a single linear decision boundary, it fails to correctly classify XOR inputs. Multilayer neural networks are required to deal with such issues.

5. Effect of Feature Normalization

Feature normalisation improves training stability and helps the perceptron converge more a bit better by ensuring that all features contribute on a similar scale during weight updates.

10. Experiment 1 - Extension

In this extension, the Single Layer Perceptron was implemented from scratch and tested on simple logical classification problems (OR, AND and NOT gates). The objective was to observe how the perceptron learning algorithm updates its weights and bias values over successive epochs until convergence.

1. OR Gate

Training Data:

$$(0, 0) \rightarrow 0, (0, 1) \rightarrow 1, (1, 0) \rightarrow 1, (1, 1) \rightarrow 1$$

Training Summary

Epoch	Weight Vector	Bias	Status
Initial	[0.0, 0.0]	0.0	Initialized
1	[0.0, 0.1]	0.0	Updated
2	[0.1, 0.1]	0.0	Updated
3	[0.1, 0.1]	-0.1	Updated
4	[0.1, 0.1]	-0.1	Converged

The perceptron successfully converged in 4 epochs and learned the OR logic. Since the OR gate is linearly separable, one linear decision boundary is enough for separation. It also takes very few epochs to achieve this.

2. AND Gate

Training Data:

$$(0, 0) \rightarrow 0, (0, 1) \rightarrow 0, (1, 0) \rightarrow 0, (1, 1) \rightarrow 1$$

Training Summary

Epoch	Weight Vector	Bias	Status
Initial	[0.0, 0.0]	0.0	Initialized
1	[0.1, 0.1]	0.0	Updated
2	[0.2, 0.1]	-0.1	Updated
3	[0.2, 0.1]	-0.2	Updated
4	[0.2, 0.1]	-0.2	Converged

The AND gate also converged successfully in 4 epochs. Similar to the OR gate, the logic is simple and linear, and does not take long to learn and arrive at a solution.

3. NOT Gate

Training Data:

$$0 \rightarrow 1, \quad 1 \rightarrow 0$$

Training Summary

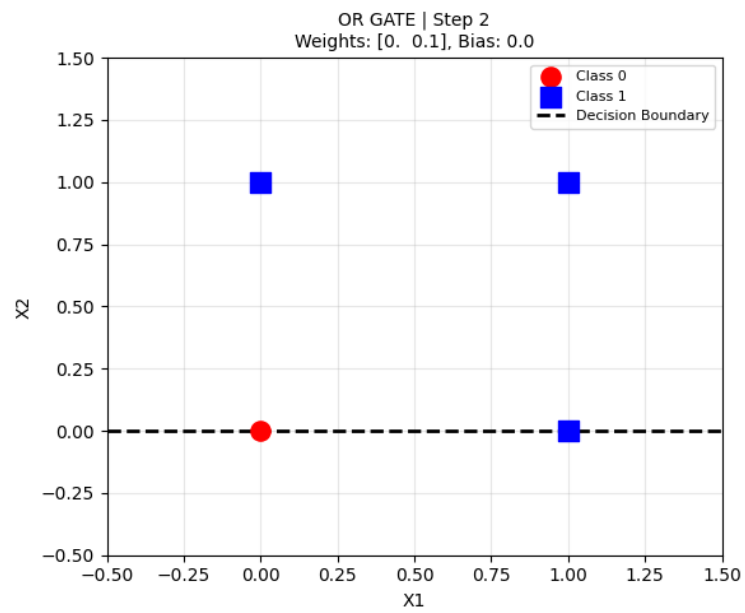
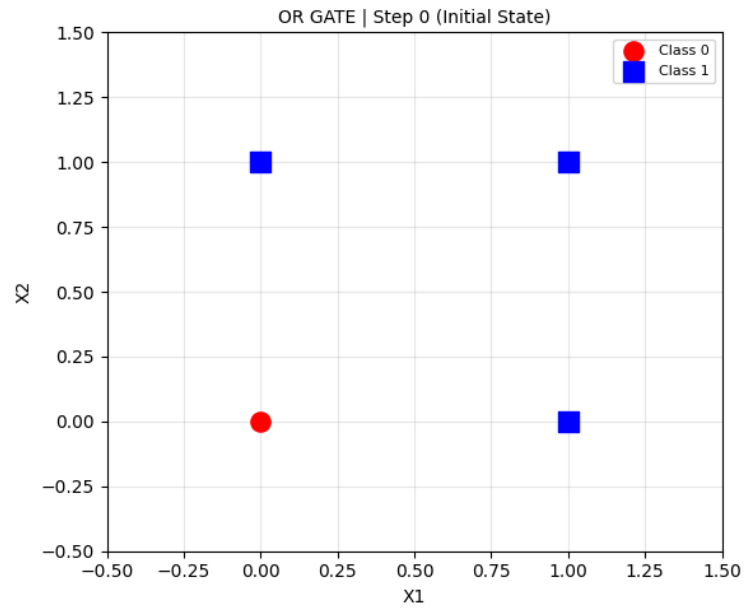
Epoch	Weight	Bias	Status
Initial	0.0	0.0	Initialized
1	-0.1	-0.1	Updated
2	-0.1	0.0	Updated
3	-0.1	0.0	Converged

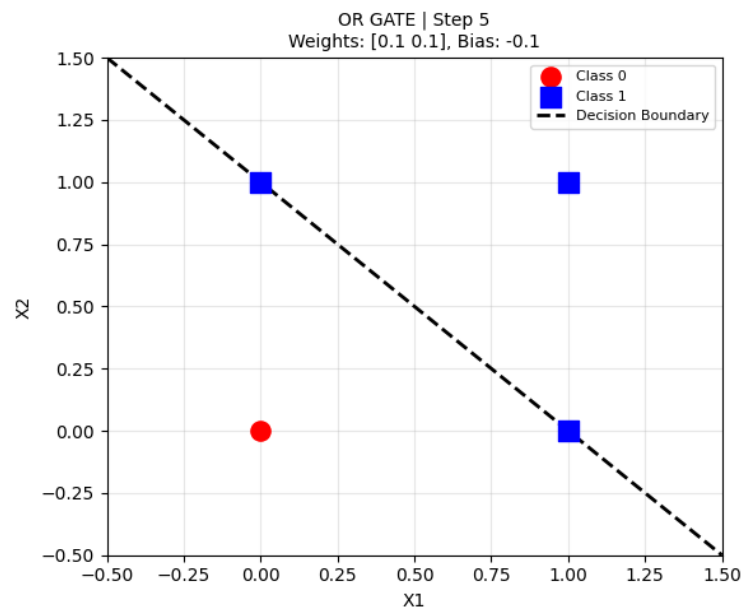
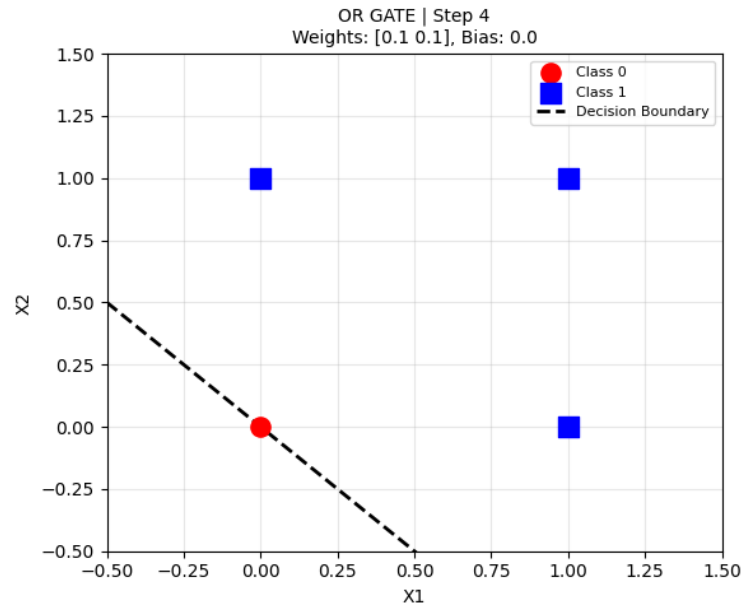
The perceptron converged in only 3 epochs for the NOT gate. The learned negative weight effectively inverts the input, demonstrating how perceptrons can represent unary logical operations using a simple linear model.

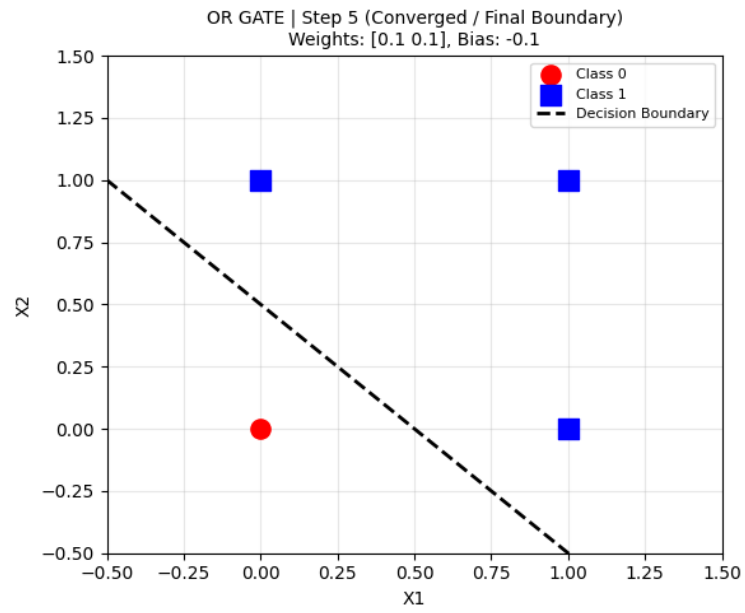
4. Plots

The figures below show the evolution of the decision boundary during training for each logic gate, from the initial state to each epoch of weight updates, up to the final converged boundary.

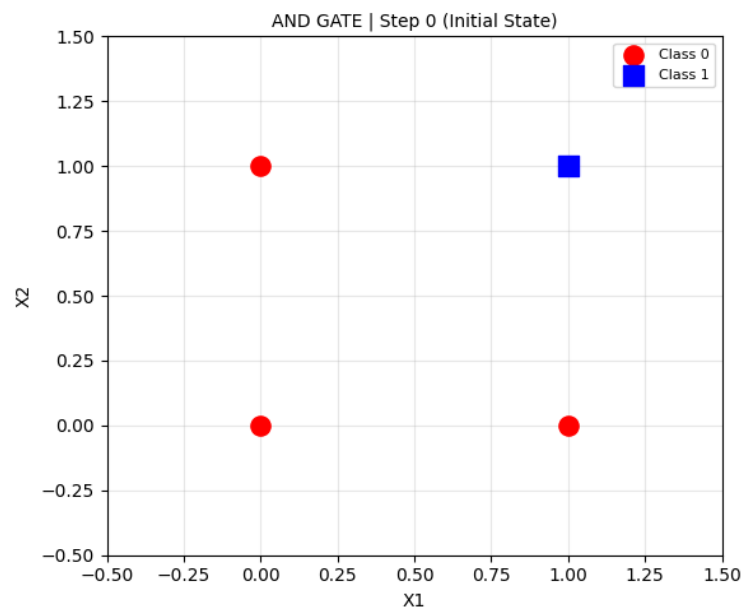
OR Gate

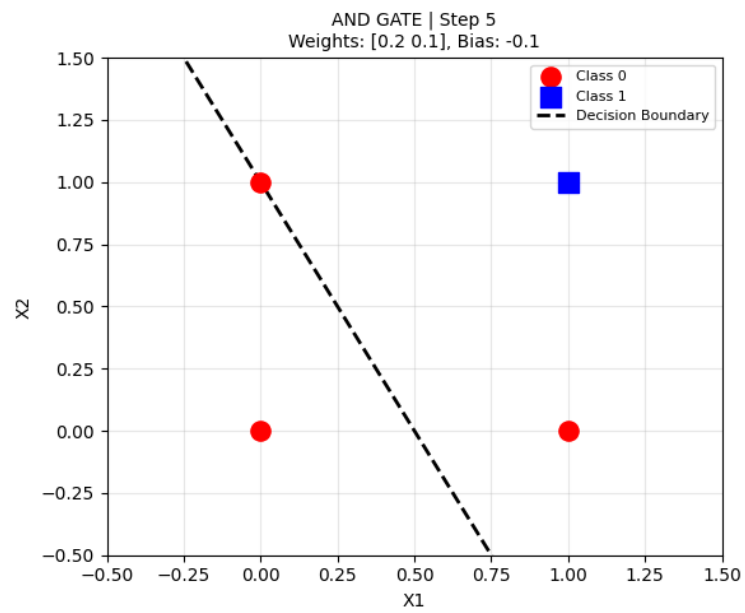
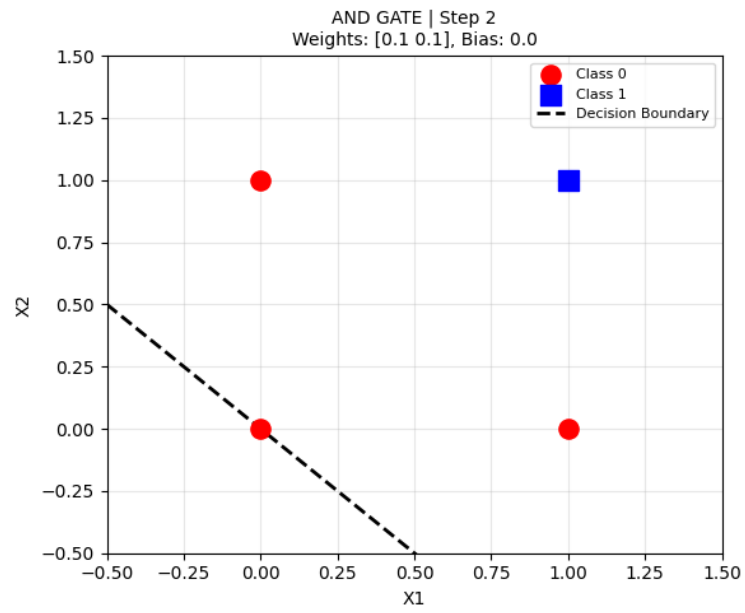


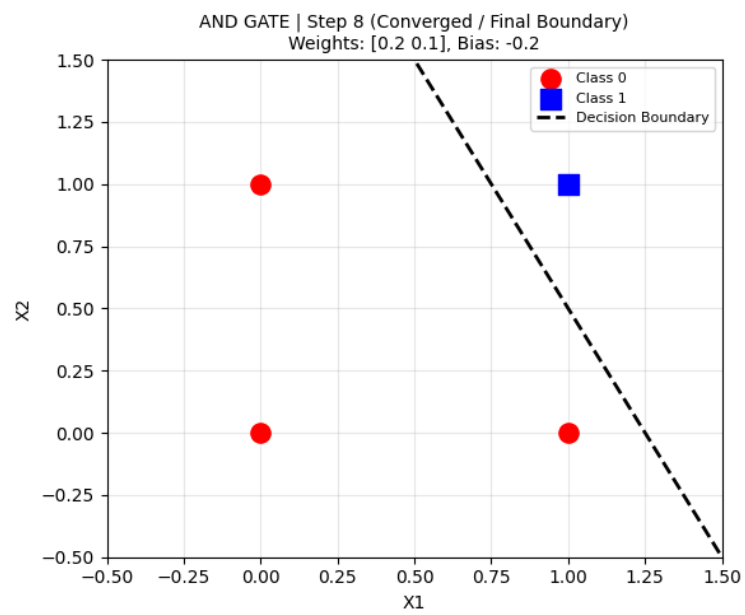
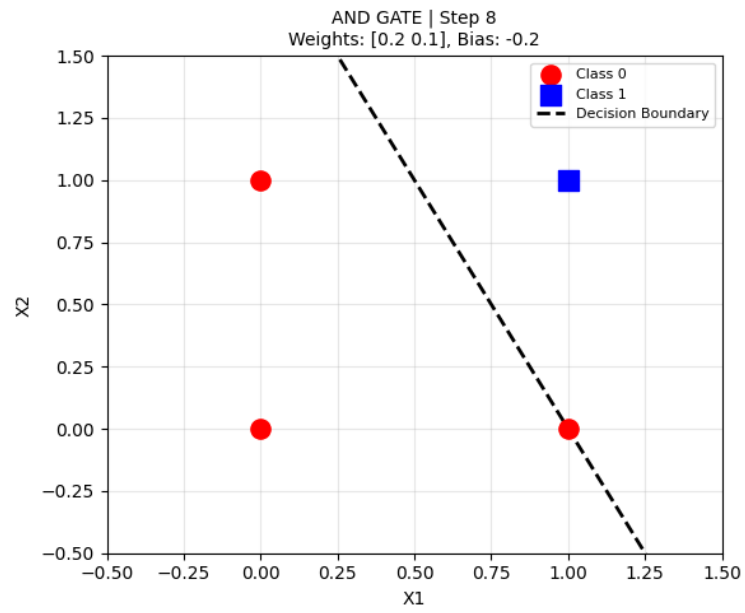




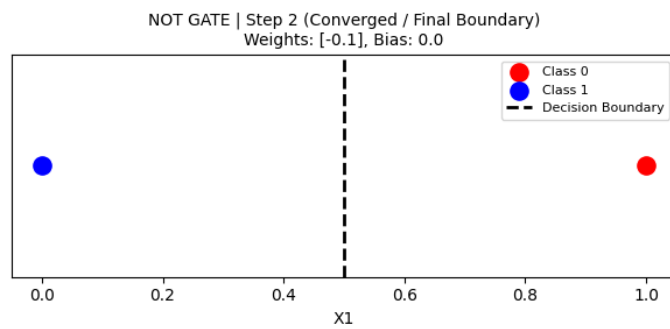
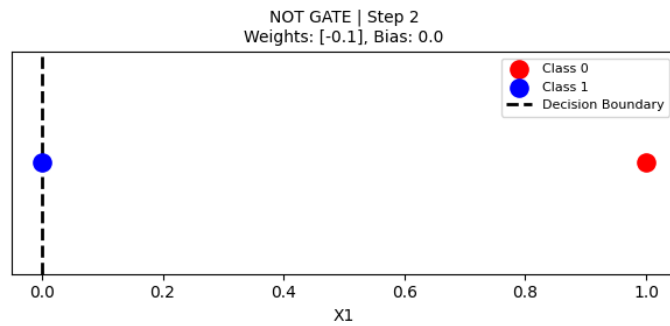
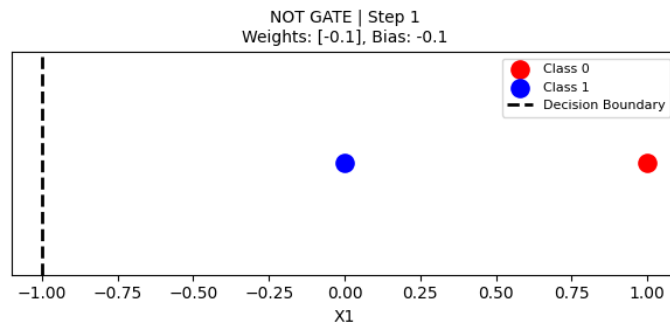
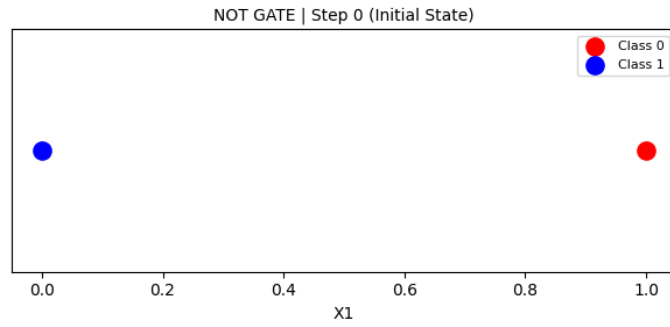
AND Gate







NOT Gate



5. Observations

- All three logical functions are linearly separable and were learned successfully by the single layer perceptron.
- The perceptron required only a few epochs to converge, highlighting the efficiency of the perceptron learning rule for simple binary classification tasks.
- Weight updates occur only when a sample is misclassified. Once all samples are classified correctly in an epoch, it has reached convergence.
- The bias term is very important. In the AND gate, a negative bias ensures that both inputs must contribute positively for the output to become 1.

11. References

1. F. Rosenblatt, "The Perceptron," *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.